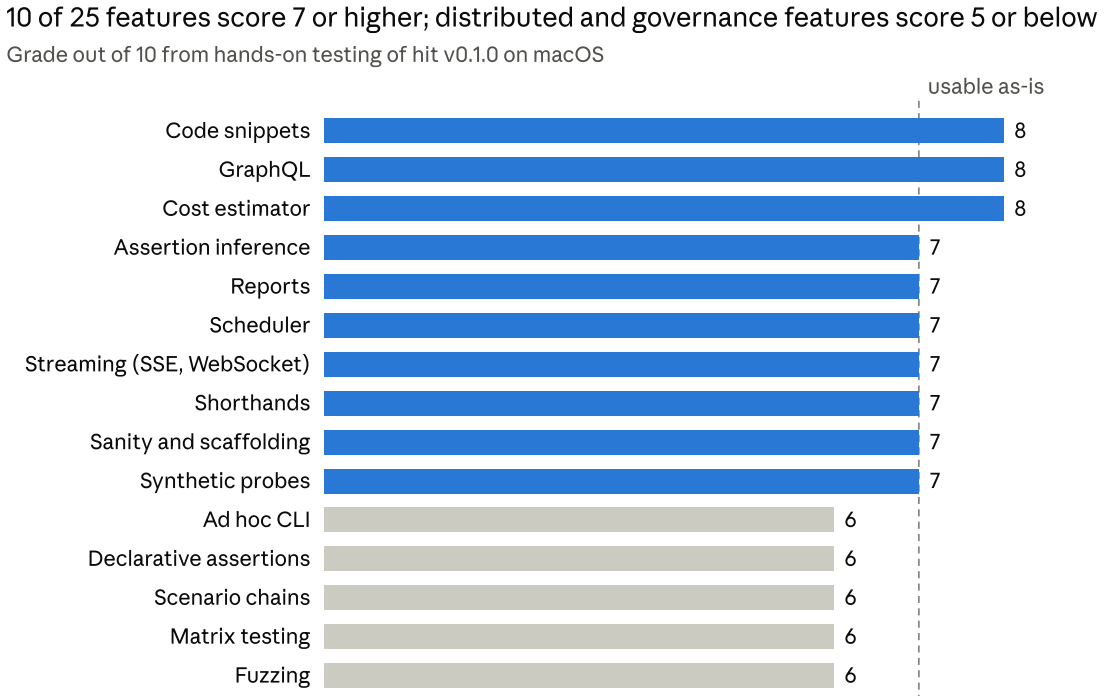
Hit Endpoint v0.1.0 Evaluation Report

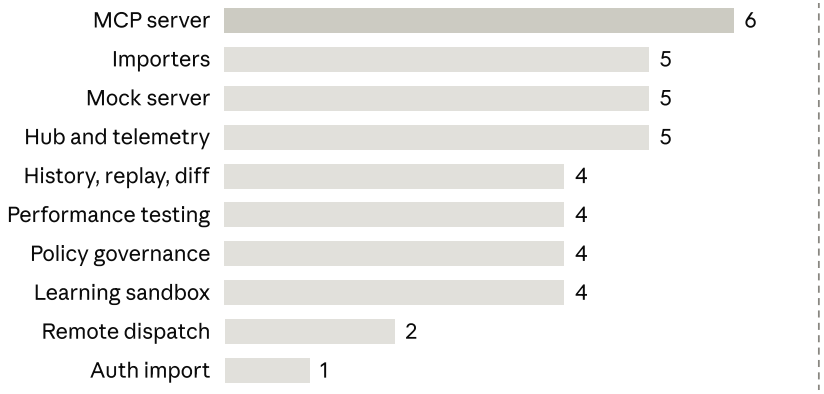
Oct 12, 2026 · @Mark Jordan

Summary

Hit Endpoint v0.1.0 is safe to adopt today for ad hoc requests, assertion inference and a small checked-in smoke suite. It is not yet safe for CI gating, load testing, governance enforcement or fleet monitoring. Across 25 graded features the mean grade is 5.7 out of 10.

The pattern is consistent. Single-process features that read a request and print a result work well. Anything involving concurrency, a second process, or a documented configuration format is unreliable, and in several places the documentation describes the repository on main rather than the released binary. One documented command, `hit auth`, does not exist in the release at all.





The bars in blue are features a team can rely on as documented. Everything at 5 or below has at least one defect that makes the feature silently wrong, unsafe, or non-functional, listed in the next section.

Bugs by severity

Critical means data loss, a crash, or a safety control that silently does nothing. High means wrong results or a secret exposed. Medium means a documented capability that does not work. Low is cosmetic or an edge case.

Severity	Feature	Bug	How to reproduce
Critical	Performance	Fatal `concurrent map writes` in the runner's secret collection. Roughly 1 run in 6 at 10 workers. Also kills the MCP server when an agent calls `hit_perf`.	Repeat `hit perf <ref> -c 10 -n 200` several times
Critical	Policy	The documented `rules:` wrapper in policy.yaml is ignored. Every file passes and the linter prints "All requests comply". Only top-level keys are read.	`hit policy` with the policy.yaml from the governance doc
Critical	Telemetry	Default publish target is the hardcoded.	`hit run <ref> --publish` with `cloud:` set in

		unregistered domain api.hitendpoint.com. The `cloud:` block in zone.yaml is ignored, so a configured zone still falls back to it. If anyone registers that domain, every unconfigured user posts run data to it.	zone.yaml
Critical	Auth import	`hit auth` does not exist in the v0.1.0 binary. Its source was committed four days after the release. README, docs and lesson 3 describe it as available.	`hit auth token x`
High	Performance	`--distribute N` and `--workers URL` drop the server's auth block. Every request returns 401 while the same request on one worker returns 200.	`hit perf <ref> -c 4 -n 20 --distribute 2`
High	Dispatch	Jobs queue and never execute. An enrolled, idle worker is not picked by hub auto-discovery and dispatched jobs time out after 2 minutes.	`hit perf worker --hub URL`, then `hit dispatch <ref> --hub URL`
High	Reports	HTML and HAR exports contain Authorization headers, captured tokens and login request bodies in plaintext. Masking applies to history and telemetry only.	`hit run <ref> --html r.html --har r.har`, grep the token
High	MCP	`hit_show` returns passwords and bearer tokens unmasked to the model. `hit_perf` runs against write endpoints with no guard.	Call `hit_show` on a login request over stdio
High	History	Replay sends the masked `[MASKED]` placeholder as the auth header and gets 401. `history save --infer` then infers `status: 401` from that failed replay instead of the stored 201.	`hit replay <id>` on an authenticated entry
High	History	Cookie headers are stored in plaintext. Authorization and `*key*` headers are masked	Send `-H 'Cookie: sid=x'`, grep `.hit/history.jsonl`

		Authorization and `key` headers are masked.	`hit/history.json`
High	Policy	`--enforce-policy` at run time does nothing. A 52ms response passed a 10ms ceiling. `require_description` and `require_auth` are never enforced under either config layout.	`hit run <ref> --enforce-policy` with `max_latency: 10ms`
High	Hub	Only ingest is authenticated. Runs, nodes, fleet and the dashboard are readable without the API key. `HIT_PROJECT_ID` is dropped. The node owner is filled from global git config, leaking a personal email. Any JSON object is accepted as a run.	`curl http://hub/api/v1/runs` with `--api-key` set
Medium	Chains	The `script:` step type from the docs does not exist, only `python:`. The hook API is `before(prep, ctx)` with no body, not the documented `run(session)`. A `print()` inside a hook corrupts the stdout JSON protocol. A header set in a before hook was not applied.	Chain with a `script:` step; hook that prints
Medium	Assertions	The README request syntax (`path:`, `assert:`, `latency: < 250ms`, `body.name`) does not exist. `path:` is silently ignored and the run fails with "has no url".	Copy the README example
Medium	Assertions	`expr` is JMESPath, so numeric literals need backticks. The docs' own example `ms < 500 && status == 200` fails with `Invalid token: tNumber`. Unknown operators become object equality and `status: twohundred` passes validate.	Run the docs example
Medium	Matrix	`{{row.field}}` is documented and unsupported. Data files resolve only relative to the request file, not the zone root. Agent output shows one NO_RESPONSE line instead of per-	Inline `matrix.rows` with `{{row.name}}`

		row results.	
Medium	Mock server	`--auth-expire` breaks login entirely for the correct password while the static demo token never expires. Sandbox password is `hunter2`, not `password` or none as two docs say. Echo is GET-only, slow takes `?ms=`, unknown routes return 401.	`hit mock 8765 --auth-expire 30s`, then login
Medium	Mock server	OpenAPI mock returns 200 with an empty body for every GET when the spec has no response schemas, including never-created and just-deleted ids.	`hit mock --openapi examples/petstore-zone/openapi.yaml`
Medium	Importers	OpenAPI import generates no request bodies, no captures and no server file, so login and create requests fail against the tool's own mock. cURL import writes `-u` passwords and bearer tokens into the collection in plaintext and drops `--data-urlencode`. Postman test scripts are pasted in as inert comments.	`hit import openapi <spec>`, run the result
Medium	Probes	The generic `webhook` alert channel is silently ignored. The daemon lists only Slack even when a webhook URL is configured, and `test-alert` never exercises it.	Probe YAML with `alerts.webhook.url`
Medium	Fuzzing	Accepted injection payloads are reported as a "Clean Bill of Health". GET requests get raw bodies instead of path or query mutations. JSON output has `anomalies: null` and no per-mutation list.	`hit fuzz <POST ref>`, note 2xx on SQL probes
Medium	Reports, perf, policy	`hit report coverage --strict`, `hit perf --check` and `hit policy --strict` have no effect on the exit code.	Run each with and without the flag

Medium	Performance	`--csv` with a relative path prints "rows written" and writes nothing. Absolute paths work.	`hit perf <ref> -n 10 --csv out.csv`
Medium	Ad hoc CLI	`-j` and `-f` do not switch the method to POST. A bodiless GET sends `Content-Type: application/x-www-form-urlencoded`. Inside a zone, ad hoc requests are labelled with the zone's absolute path plus `/adhoc`. Repeated `-H` keeps only the last.	`hit URL -j '{}'` inside a zone
Medium	Sanity, scaffolding	`hit init` and `hit wizard` are separate code paths. Only the wizard produces the layout the README shows. Sanity validates the default server only. `init .` inside a zone silently nests a zone.	Compare `hit init a` with `hit wizard b -y`
Medium	Learning sandbox	Lesson outputs do not match the mock server. The grader pattern-matches global history, so lesson 6 passes before any exercise and unrelated requests count.	`hit learn verify all` on a fresh zone
Low	Cost	A call count of 0 or -5, a failure rate of 150 percent and -1 retries are accepted. The 150 percent case yields an expected cost above the worst case.	`hit cost -n 10 --unit-price 1 --failure-rate 150%`
Low	Streaming	JSON timings (`ttft` , `handshake_latency`) are raw nanoseconds with no unit. The docs' WebSocket example host no longer resolves.	`hit sse URL --json`
Low	History	Bodies are truncated at 4000 characters and the diff does not warn, so large JSON diffs degrade to one line per side. No ignore list for volatile headers. Indexes are 1-based while the help text uses 0.	`hit diff 2 1` on two large responses
Low	Shorthands	No collision check. A shorthand named `code`	`hit shorthand set code URL`

		saves and is unreachable. A shorthand matching a collection folder is accepted silently.	
Low	Perf, scheduler	Histogram prints duplicate bucket labels. Scheduler ticks have no sub-second timestamps. Failure exit codes are 2 rather than 1.	Any `hit perf` run

Core CLI

Ad hoc requests, 6 of 10

The everyday path works: method, headers, query, JSON and form bodies, files, basic and bearer auth, redirects, gzip, the body/code/time/headers output modes, and `--status` as an exit-code gate. Verbose mode shows outgoing headers.

- Body flags do not switch the method. `hit URL -j '{...}'` sends a GET with a body.
- A bodiless GET sends a form content type.
- Inside a zone the banner and history label is the zone's absolute path plus `/adhoc`, which hides the URL.
- Repeated `-H` headers keep only the last one.
- The `--auth` error message omits the `cookie:` form the source accepts. There is no API-key form.
- Default user agent is `Go-http-client/1.1`. No timeout flag. No stdin body. URLs need a scheme.

Declarative assertions, 6 of 10

Every documented operator worked when written correctly. Named tests, expected-versus-actual failure output with file paths, exit code 1 and the compact agent format are all good.

- The README request syntax does not exist. `path:`, `assert:`, `latency:` and `body.name` are ignored or rejected.
- `expr` needs JMESPath backticks around numeric literals. The docs' examples fail.
- No schema validation of test blocks. Unknown operators become object equality.

Assertion inference, 7 of 10

`hit assert` produced a typed contract for a nested Discogs release in one command. `--strict`, `--save` and `--append` work. The latency bound is max(500ms, 2.5 times observed). In non-strict mode it still pins enum-looking strings such as `kind: dog` to exact values, which will go stale.

Importers, 5 of 10

Local file auto-detection works. A missing file is reported as "unknown import kind", which misleads.

- OpenAPI: no request bodies, no captures, no server file from the spec's servers list, path params left as unresolved `{{id}}`. A richer remote spec produced captures.
- Postman: folders, auth, variables and the environment-to-server-plus-secrets split all import cleanly. Test scripts become inert comment lines. Import spills `console.log` lines to the terminal.
- cURL: `-u` plus a Bearer header produces a confused auth block. `--data-urlencode` is dropped. Secrets are written in plaintext to the collection, which the tool's own policy linter then flags.

History, replay and diff, 4 of 10

The log is append-only, capped at 1000 entries, kept per zone and globally. Semantic diff of status, latency, size, headers and canonical JSON is a good design.

- Cookie headers are stored in plaintext.

- Replay of an authenticated request sends the masked placeholder and gets 401. `history save --infer` infers from that failed replay.
- Replay re-sends POST and DELETE with no confirmation or dry run.
- Bodies truncate at 4000 characters with no warning in diff.
- No ignore list for volatile headers, so every diff flags date, cf-ray and rate-limit counters.

Suites and simulation

Scenario chains, 6 of 10

Request steps with variable overrides, captures that persist to `.hit/state` and are reused by later runs, `set`, `sleep`, `repeat`, `replace_tests`, nested flows, `continue_on_fail` and `--keep-going` all behaved as described. Captured tokens are masked in output.

- The documented `script:` step does not exist. Only `python:` is accepted.
- The Python API is `before(prep, ctx)` with method, URL, headers and params. There is no body, so the documented HMAC-signing example cannot be written. A header set in `before` was not applied in testing.
- Any `print()` in a hook corrupts the stdout JSON protocol.
- A flow that stops on a hard step failure still prints "all passed". The exit code is correctly 1.
- README says `chains/`, docs say `flows/`. Only `chains/` is discovered.

Matrix testing, 6 of 10

Inline rows, CSV and JSON files run, the scorecard is clear, and JUnit emits one case per assertion per row.

- `{{row.field}}` is documented and does not resolve. Only bare `{{field}}` works.

- Data files resolve relative to the request file only.
- Agent output reports the matrix as a single NO_RESPONSE line.

Mock server, 5 of 10

Chaos features are the strong part. Flaky percentages, forced status codes, rate limiting with correct Retry-After and X-RateLimit headers, fixed latency, jitter, payload corruption and per-request chaos headers all worked as documented. Bad flag values are rejected clearly.

- The sandbox password is `hunter2`. Lesson 3 says `password` and the mock doc says username alone suffices.
- Documented routes differ from the server: slow takes `?ms=`, echo is GET-only, pets is not paginated, PUT is unsupported.
- Unknown routes return 401 because auth runs before routing.
- `--auth-expire` rejects the correct password while the hardcoded demo token never expires.
- The OpenAPI mock returns 200 with an empty body for everything when the spec omits response schemas, including deleted and never-created ids. With a schema-rich spec synthesis was reasonable.

Performance testing, 4 of 10

The single-node engine produces good numbers. About 8,500 requests per second on one worker, an RPS cap within 1 percent of target, correct threshold evaluation, 4xx counted as errors, and chain load tests with per-worker state.

- Intermittent `fatal error: concurrent map writes`, 1 run in 6 at 10 workers.
- `--distribute` and `--workers` drop server auth. All requests return 401.
- `--csv` with a relative path writes nothing while claiming success.
- `--check` makes no difference to the exit code.
- Duplicate histogram bucket labels. No warning before load-testing a write endpoint.

Quality gates and output

Fuzzing, 6 of 10

All six mutation categories fire, verbose mode shows each payload and result, `--fail-on-5xx` exits non-zero, and the SARIF export is valid 2.1.0. Against the chaos mock it reported crashes correctly.

- Accepted injection payloads are reported with a check mark and a "Clean Bill of Health". The server stored SQL injection strings and ten-thousand-character names as pet kinds with 201.
- GET requests get raw bodies and corrupted headers. Path and query parameters are never mutated.
- JSON output has `anomalies: null` and no per-mutation list.

Code snippets, 8 of 10

Python, JavaScript, PHP and Go output are idiomatic and include timing and the extracted field. Snippets from a zone request resolve variables and auth. `--all` emits all four. `show --curl` round-trips cleanly but always appends `-L`.

Reports, 7 of 10

JUnit has one suite per request and one case per assertion. HAR is valid 1.2. The HTML report is a single self-contained file with no CDN dependencies. Coverage against an OpenAPI spec lists untested operations and undocumented requests. The latency trend report and failure webhook both work.

- HTML and HAR exports contain the Authorization header, captured tokens and the login password body in plaintext.
- `coverage --strict` has no effect. Two untested operations still exit 0.

- HAR timings populate only `wait`.

Policy governance, 4 of 10

When it runs, detection is good. It caught a JWT in a header, a live-style API key, a plaintext basic password, a literal bearer token, `verify: false`, a latency ceiling breach and a missing body assertion, each with a fix hint.

- The documented `rules:` wrapper is silently ignored. With the documented layout every file passes.
- `require_description` and `require_auth` produce no violations.
- `--enforce-policy` at run time does nothing.
- `--strict` is identical to the default. JUnit counts are inconsistent (5 tests, 8 failures).

Scheduler, 7 of 10

Intervals, run counts, duration caps, delayed starts, status and regex expectations, zone references with their assertions, JSON output and history recording all work. Ten ticks at 100ms completed in 0.92 seconds. Failures exit non-zero. Tick timestamps lack sub-second precision.

Protocols, tooling and fleet

GraphQL, 8 of 10

Inline queries, variables inline and from files, `--json`, `--fail-on-errors` with a clear error listing, introspection, SDL export and `body.graphql` in a zone request all worked against a public schema. Without `--fail-on-errors` a response full of GraphQL errors exits 0.

Streaming, 7 of 10

SSE handled named events, ids, multi-line data, comments, `--event` filtering, `-n` limits, headers and POST payloads. WebSocket connected to a public echo, sent and received, and `--expect` passed and failed correctly. JSON timings are raw nanoseconds with no unit. The docs' WebSocket example host no longer resolves. Time-limited SSE runs report no close reason.

Cost estimator, 8 of 10

Arithmetic verified by hand for graduated tiers, volume cliffs, flat rate, package rounding, LLM token presets and the Twilio pricing file. Retry amplification uses the geometric series correctly. Budget gates compare the expected scenario and exit non-zero. Input validation is the only gap: zero or negative call counts, failure rates above 100 percent and negative retries are accepted.

Auth import, 1 of 10

`hit auth token`, `hit auth cookie` and `hit auth show` return "unknown command" in the v0.1.0 binary. The source was committed on 2026-09-28, four days after the 2026-09-24 release. The README command table, the zones doc and lesson 3 describe it as available.

Shorthands, 7 of 10

Set, list, get, remove, JSON, headers, zone references with a server, global scope and use from the scheduler all work. There is no collision check with command names or collection folders.

Sanity and scaffolding, 7 of 10

`hit sanity` checks servers, variables, captured state, gitignore hygiene, requests without tests, chain resolution and live reachability, with JSON output and a non-zero exit when the server is down. `hit init` and `hit wizard` are different code paths; only the wizard produces the documented layout. Sanity validates the default server only.

`init .` inside a zone nests silently.

MCP server, 6 of 10

The stdio server speaks JSON-RPC correctly, reports protocol 2024-11-05, exposes six tools with input schemas, sets `isError` on failed assertions, and works from outside the zone with `-w` or `-z`. Malformed lines are ignored and unknown methods get a proper error code.

- The server died mid-session during `hit_perf` against a write endpoint, after creating 5 of 20 records. It inherits the perf race.
- `hit_show` returns passwords and bearer tokens unmasked.
- No guard on destructive tools. Resources and prompts endpoints are not implemented.

Hub and telemetry, 5 of 10

Ingest works behind the API key. The dashboard is one 50KB page with no external dependencies. Stored payloads contain no response bodies or secrets. Publishing via the flag, `HIT_PUBLISH`, and `HIT_CLOUD_URL` all delivered runs.

- Only writes are authenticated. Runs, nodes, fleet, stats and dashboard are open.
- The `cloud:` block in zone.yaml is ignored and publish falls through to api.hitendpoint.com.
- `HIT_PROJECT_ID` is dropped. The node owner is filled from global git config.
- Any JSON object is accepted as a run. Unknown API paths return the dashboard HTML with 200.

Remote dispatch and distributed load, 2 of 10

The worker daemon starts, enrols, heartbeats and answers its health endpoint. Hub auto-discovery then reports no active workers. Dispatched jobs queue and time out after two minutes with the worker idle. Distributed perf via an explicit worker list drops auth. Per-

worker throughput printed as 28,342 req/s while the overall figure was 39.8.

Synthetic probes, 7 of 10

One-shot runs evaluate every region, apply the consensus threshold and latency SLA, and exit non-zero on an incident. The daemon moved correctly through healthy, incident and resolved, fired exactly one trigger and one resolution with no duplicate pages, and enrolled in the hub. `test-alert` delivered correctly shaped PagerDuty Events v2, Slack Block Kit and Opsgenie payloads with `$env:` secrets resolved.

- The generic `webhook` alert channel is silently ignored.
- Regions are simulated locally with no indication in the output.
- The hub has no probe or incident endpoints, so enrolment records a node and nothing else.

Documentation mismatches

The README on main describes the repository, not the v0.1.0 download. Each row is a place where a user copying the docs will fail.

Where	Docs say	Binary does
README, zones doc, lesson 3	`hit auth token`, `hit auth cookie`, `hit auth show`	Unknown command
README request example	`path:`, `assert:`, `latency: < 250ms`, `body.name`	Needs `url:`, `tests:`, `max_ms`, `json:`
README scaffold	`hit init` creates chains, shorthands, production server, secrets examples	`init` creates 5 files; only `wizard -y` creates that layout
README layout vs flows doc	`chains/` vs `flows/`	Only `chains/` is discovered
Flows doc	`script:` step, hooks define `run(session)` with `session.body`	Only `python:`; hooks define `before(prep, ctx)` with no body

Assertions doc	`expr: "ms < 500 && status == 200"`	Needs backticks: `ms < `500``
Matrix doc	`{{row.field}}` supported; files relative to request or zone root	Only `{{field}}`; request-relative only
Policy doc	Rules nested under `rules:`	Top-level keys only; nested form silently passes everything
Mock doc	`GET /slow?delay=500ms`, `POST /echo`, paginated pets, login with username alone	`?ms=`, GET-only echo, no pagination, password `hunter2`
Lesson 2	Unauthenticated `POST /pets` returns 201	Returns 401
Lesson 3	Login returns `{"token": "mock-jwt-token-admin"}`, capture with `json.token`	Returns `{"access_token": "demo-token-123"}`
Telemetry doc	`cloud:` block in zone.yaml configures the endpoint	Ignored; falls back to api.hitendpoint.com
Telemetry doc	Cookie header is scrubbed	History store leaves Cookie in plaintext
Probe doc	`alerts.webhook.url` generic channel	Ignored by daemon and test-alert
Streaming doc	`wss://echo.websocket.events` example	Host does not resolve
Help text	`hit assert 0 --append`	Indexes are 1-based
Lesson 7, perf doc, policy doc	`--strict`, `--check` as CI gates	No effect on exit code

Recommendations

For a team adopting the tool

Use it now for ad hoc requests, `hit assert` to snapshot endpoint contracts, snippets, GraphQL, the cost model, and a checked-in smoke suite run by hand or in a non-blocking CI step. Pin the telemetry endpoint with `HIT_CLOUD_URL` or [illegible] publishing off. Treat

CI step. Pin the telemetry endpoint with `HIT_CLOUD_URL` or leave publishing off. Treat `.hit/history.jsonl` as a log that holds cookies and response bodies, and clear it before sharing a machine.

Do not use `hit perf` in CI until the race is fixed. Do not rely on `hit policy` as a gate. Do not attach HTML or HAR reports to tickets without scrubbing them. Do not expose the MCP server to an agent that can reach production.

For the maintainer, in order

1. Fix the `concurrent map writes` race in the runner. It affects perf, distributed perf and the MCP server.
2. Make the policy parser accept the documented `rules:` layout or fail loudly on an unrecognised one. Wire up `--enforce-policy`, `require_description` and `require_auth`.
3. Remove the hardcoded api.hitendpoint.com default, or register the domain. Honour the `cloud:` block in zone.yaml.
4. Cut a release that matches the README, or mark unreleased features in the README. `hit auth` is the visible case.
5. Pass server auth through to distributed workers. Make dispatch and hub auto-discovery agree on what an enrolled worker is.
6. Apply the same masking to HTML, HAR and MCP `hit_show` output that history and telemetry already get. Mask Cookie in history.
7. Make replay re-apply auth from the server config instead of sending the masked placeholder, and have `save --infer` use the stored response.
8. Align the docs with the binary on the items in the mismatch table. The hook API and the mock sandbox credentials are the ones that break the tutorials.
9. Report accepted injection payloads in fuzz output, and mutate path and query parameters on GET.

Methodology

Method

Binary: hit 0.1.0, darwin/arm64, installed by the project's install script to `~/.local/bin/hit`. Source read from hit-endpoint/hit-endpoint on main as of 2026-10-12, 18 commits from one author, release v0.1.0 published 2026-09-24. Machine: macOS, Go 1.22.12 runtime reported by the binary, Python 3.14 for hooks and helper servers.

Each feature was exercised with the commands in its own doc page, then with edge cases, against the bundled mock server on localhost, the OpenAPI mock built from the bundled Petstore spec, local Python servers for SSE, telemetry and alert capture, and read-only calls to the public Discogs API and a public GraphQL schema. Where behavior looked wrong the relevant Go source was read to confirm whether it was a defect or a documentation error. Exit codes were measured without pipes. The perf crash rate is from six consecutive runs.

Not tested: Windows and Linux builds, real multi-machine load, the Opsgenie and PagerDuty live APIs beyond payload shape, and the `hit learn` lessons beyond the first three.